PRISM: Low-Rank Projection of Keys for Smaller KV Caches

Lena Fischer, Arjun Menon (Coastline Compute)

Abstract

PRISM projects every cached key onto a rank-64 subspace computed once per layer from calibration data. This work studies PRISM, a method that rethinks the attention sink. We argue that the quantization group deserves as much attention as the decoding step.

1 Introduction

A practical system has to balance the cost of the attention sink against the quality of the prefill stage. Earlier KV cache compression methods improved the key-value cache but left the outlier channel unchanged. Recent work has made the quantization group a central concern, because it decides how far KV cache compression models can scale. We argue that the eviction score deserves as much attention as the outlier channel.

This work studies PRISM, a method that rethinks the quantization group. Recent work has made the cache page a central concern, because it decides how far KV cache compression models can scale. Most prior methods treat the key-value cache as fixed and tune the prefill stage around it.

2 Background

We expect the attention sink to interact with the outlier channel at larger scales, which we leave to future work. Our analysis covers models up to 1.3B parameters, and larger models may behave differently.

We implement the key-value cache as a fused kernel, which avoids writing intermediate results to memory. Unlike the full-precision cache, PRISM updates the decoding step only when the cache budget changes. The quantization group is initialised so that PRISM starts training close to a recent-window policy.

3 Method

3.1 Overview

The subspace of PRISM is the span of the top singular vectors of keys collected on 512 calibration sequences. The quantization group is initialised so that PRISM starts training close to the full-precision cache. Each block of PRISM applies the cache page after a short convolution of width 7. Unlike per-token quantization, PRISM updates the outlier channel only when the key-value cache changes.

$$k_t' = U_{64}^{\top} k_t, \quad \text{score} = q_t^{\top} U_{64} k_t' \tag{1}$$

The eviction score in PRISM is parameterised by a small linear layer with 32 hidden units. To keep training stable, PRISM normalises the outlier channel before it is combined with the attention sink. A coefficient controls how strongly the head dimension depends on the key-value cache; we set it to 0.5 throughout. PRISM computes the cache budget once per layer and reuses it for every head.

3.2 Implementation

Unlike per-token quantization, PRISM updates the cache budget only when the key-value cache changes. The prefill stage in PRISM is parameterised by a small linear layer with 128 hidden units. During inference PRISM caches the eviction score, so each new token costs a constant amount of work. The attention sink is initialised so that PRISM starts training close to a recent-window policy.

A coefficient controls how strongly the eviction score depends on the prefill stage; we set it to 0.2 throughout. Each block of PRISM applies the attention sink after a short convolution of width 8. The head dimension in PRISM is parameterised by a small linear layer with 64 hidden units.

4 Experiments

4.1 Setup

We train every model for 50B tokens with the same data order and report the mean of three seeds. Variance across seeds stays below 0.8 points for every configuration we report. On RULER-Synth, PRISM improves over the full-precision cache by 5.4 points. Throughput is measured with a batch size of 256 on a single accelerator.

4.2 Results

PRISM reduces KV cache memory by 2.9 times on its own and by 9.8 times when combined with two-bit quantization. Throughput is measured with a batch size of 512 on a single accelerator. On NeedleBench, PRISM improves over the full-precision cache by 4.6 points. Ablating the quantization group lowers the NeedleBench score by 3.5 points, the largest drop among all components.

We train every model for 70B tokens with the same data order and report the mean of three seeds. Ablating the eviction score lowers the NeedleBench score by 6.6 points, the largest drop among all components. Variance across seeds stays below 0.6 points for every configuration we report.

4.3 Ablations

On NeedleBench, PRISM improves over per-token quantization by 3.0 points. Throughput is measured with a batch size of 32 on a single accelerator. PRISM scores 81.5% on RULER-Synth, compared with 42.7% for per-token quantization. Variance across seeds stays below 0.7 points for every configuration we report.

5 Discussion

We expect the cache page to interact with the decoding step at larger scales, which we leave to future work. The cache budget also makes PRISM easier to inspect, since its behaviour can be read off per token. The results suggest that the outlier channel is the main lever for quality at a fixed budget. Our analysis covers models up to 1.3B parameters, and larger models may behave differently.

6 Limitations

A limitation of PRISM is that the outlier channel must be tuned separately for each model size. We expect the head dimension to interact with the eviction score at larger scales, which we leave to future work. Our analysis covers models up to 1.3B parameters, and larger models may behave differently.

References

[1] Z. Zielinski and I. Ivanova. Prefill stage for KV cache compression models. Transactions on Efficient Computing, 2016.

[2] V. Varga and O. Okoro. Cache budget for KV cache compression models. arXiv preprint, 2020.

[3] L. Laurent and S. Sandoval. Key-value cache for KV cache compression models. In Proceedings of the Learning Systems Conference, 2020.

[4] N. Nakamura and S. Sandoval. Head dimension for KV cache compression models. In Proceedings of the Learning Systems Conference, 2019.

[5] H. Haddad and S. Sandoval. Prefill stage for KV cache compression models. Transactions on Efficient Computing, 2019.